Wikipedia API Final report

Team 3: Alanna Cao, Ellen Fang, Tianyin Wang

GitHub Repository: https://github.com/cmu-api-design/team3-f24
API Documentation: https://cmu-api-design.github.io/team3-f24/wikipediaapi.html

Project Motivation and Overview

The Wikipedia Python Wrapper API was designed to address limitations in the original MediaWiki API wrapper, enhancing its functionality, usability, and adaptability. While the original wrapper provided basic access to Wikipedia data, it lacked support for advanced search and extraction features, robust error handling, concurrent operations, and overall adherence to modern API principles. This project aimed to create a more comprehensive, flexible, and user-friendly wrapper by expanding its functionality and improving its design, particularly within the scope of supporting fetching and extracting operations on Wikipedia pages. Our extended and redesigned API focuses on:

- Improved usability: Enhanced Pythonic design, intuitive and custom error handling, and consistent naming conventions
- Expanded functionality
    - Advanced search capabilities with flexible sorting and filtering (title, text, multilingual, location-based, and prefix-based)
    - Support for Wikipedia's revision property to track and summarize page updates
    - Infobox extraction for structured data
    - Media handling for retrieving images, audio, and video metadata
    - Location, length, template, and redirect attributes extracted from wikipedia page metadata
- Concurrent Access: Optimized for fetching and analyzing large sets of Wikipedia pages in parallel
- Multilingual Support: Introduced a Language Enum to eliminate errors in handling multiple language versions of pages
- Design flaws fixing

These enhancements allowed us to implement a wide range of use cases, including creating educational flashcards, generating high-level reports, and monitoring Wikipedia page revisions.

Use Cases and Applications

Our use cases were realized through user interviews conducted to better understand what Wikipedia users would prioritize in a search and fetching-focused MediaWiki wrapper. We designed the following use cases with the goal of demonstrating how our improved wrapper bridges the gap in the original wrapper by supporting diverse real-world applications.

1. **Educational Flashcards (client_code/flashcards.py)**:

This use case was motivated by the fact that many educators and students rely on Wikipedia for introductory information on topics. Creating study materials like flashcards from this data could save time and enhance learning. To implement this use case, we leveraged the newly implemented infobox extraction functionality to fetch structured data (e.g., key facts or attributes of a topic) from Wikipedia pages. This data was then parsed and transformed into

educational flashcards, including fields like definitions, attributes, or key events. Overall, the client code for this use case demonstrates our API's ability to parse and organize structured data efficiently, and showcases the flexibility and precision of the infobox parsing feature, justifying our decision to add this feature to the original wrapper.

2. **High-Level Reports (client_code/report.py)**:

Since Wikipedia users often require structured summaries of Wikipedia pages for documentation or reporting purposes, and the original wrapper lacked the functionality to generate comprehensive reports, we decided to write client code demonstrating how our improved wrapper achieves this. To implement this, we used the enhanced WikipediaPage class to fetch metadata, sections, infoboxes, and other structured data. A very simple template was designed to organize this extracted data into a high-level report, including page and section titles, summaries, categories, links, and media metadata. This client code highlights the wrapper's capability to combine metadata, textual content, and hierarchical structures in a single query.

3. **Multilingual Comparison (client_code/LLM_client_code.py)**:

Motivated by the multilingual nature of Wikipedia, where the same topic could have different cultural perspectives or details depending on the language, we designed a use case to compare knowledge across languages and determine if the content of Wikipedia pages are influenced by the context of the language it's in. Since our improved wrapper includes a Language Enum to eliminate potential errors in specifying language codes and simplify access to multilingual pages, we made use of this improved feature in our client code for this use case. To implement the use case, we first fetch and compare the content from different language versions of the same Wikipedia page. By integrating this functionality with an LLM-based (gpt-4o mini) analysis, we can identify if there are any major differences in tone, detail, or focus between language versions. This use case highlights the wrapper's ability to facilitate and contribute to cross-cultural research and advanced comparative analysis on Wikipedia pages.

4. **Page Revisions Monitoring (client_code/LLM_revisions_client_code.py)**:

Wikipedia's dynamic nature means that pages are frequently updated, often with significant edits that researchers, journalists, and students may want to track. To address this, we enhanced the WikipediaPage class by adding a revisions property that abstracts away the complexities of interacting with the MediaWiki revisions API. Our client code demonstrates how to fetch and analyze page revision histories, identifying substantive changes while filtering out minor edits. Additionally, it supports summarizing and comparing revisions to highlight key differences, and includes real-time monitoring functionality to detect and summarize new updates as they occur. This use case demonstrates the wrapper's strength in handling dynamic content and provides tools for maintaining accurate and up-to-date information.

5. **Concurrent Page Access (client_code/concurrent_load.py)**:

Large-scale applications, such as academic research or bulk data analysis, require efficient access to multiple Wikipedia pages. The original wrapper handled requests sequentially, making such operations time-consuming and less scalable. To address this, we extended the WikipediaClient class to support concurrent access using threading. Our client code demonstrates how multiple pages can be fetched by title or ID simultaneously, with a configurable max_threads parameter to balance performance and resource use. Thread safety was ensured through proper locking mechanisms, preventing race conditions during concurrent requests. This use case not only highlights the improved wrapper's scalability but also its potential to power data-intensive workflows involving thousands of pages.

Design Summary

Core Classes

- **WikipediaClient**: Handles querying pages and interactions with the API.
- **WikipediaPage**: Represents the fields of a Wikipedia page and manages data fetching and caching.
- **WikipediaPageSection**: Represents sections of a page, encapsulating text, subsections, and media.
- **MediaObject**: Encapsulates metadata for media files associated with a Wikipedia page section.

Utility Classes and Enums

- **ExtractFormat Enum**: Specifies extraction format (e.g., WIKI, HTML).
- **Namespace Enum**: Categorizes pages by namespace.
- **MediaType Enum**: Classifies media files (e.g., image, video).
- **Language Enum**: Enumerates supported languages.
- **Sorting Enums**: Define sorting orders for search results.

Error Handling

- Custom exceptions (e.g., InvalidInputError, PageNotFoundError) simplify troubleshooting for API clients.

Key Design Decisions / Improvements

1. **Caching**:

Caching was implemented at the page level to allow for granular control over the retrieval and storage of data while ensuring the content remains up-to-date. By caching at this level, frequently accessed pages can be quickly retrieved without the need for redundant API calls, significantly improving performance and reducing load on the MediaWiki API. An alternative considered was client-level caching, which would have centralized the caching logic for all operations but would have sacrificed the flexibility to manage content freshness for individual pages. Our approach strikes a balance between performance and modularity, making it easier for users to work with frequently queried pages.

2. **Class Design**:

Separation of Concerns

One of the major shortcomings of the original wrapper was the lack of separation of concerns, with methods such as links and categories being duplicated and misplaced in the base Wikipedia class rather than being organized within relevant classes. We addressed this by adopting a modular class design, distributing responsibilities among distinct classes such as WikipediaClient, WikipediaPage, WikipediaPageSection, and MediaObject. For instance, page-specific metadata and content fetching were encapsulated within WikipediaPage, while section-level organization and media data were handled by WikipediaPageSection and MediaObject, respectively. This separation improves maintainability and scalability, as functionality can be added or modified without introducing dependencies across unrelated parts of the API. While a single-class design was considered for simplicity, it was ultimately deemed less maintainable and scalable, especially as the API expanded to support advanced features.

Mutability & Encapsulation

The original wrapper API exposed multiple public fields, creating risks of accidental modifications and breaking changes. We encapsulated these fields, making them either private or protected and providing getter methods where necessary. This design improves data integrity and ensures that the API adheres to modern object-oriented principles. By encapsulating mutable fields, users interact with well-defined interfaces rather than directly modifying attributes, reducing the likelihood of unexpected behavior.

3. **Flexibility with kwargs**

To simplify extensibility and accommodate evolving MediaWiki API properties, we adopted a Pythonic approach of using **kwargs in key methods such as search_pages. This design decision allows users to specify additional parameters dynamically without being constrained by a rigid method signature. While a builder pattern was considered for its explicit structure and control, it was ultimately rejected as it would have introduced unnecessary

complexity and been less adaptable to frequent MediaWiki API updates. The use of **kwargs aligns with Python best practices, promoting both flexibility and usability.

4. **Custom Error Handling**

The original wrapper provided limited error feedback, often returning generic or cryptic error messages. We introduced custom exceptions such as InvalidInputError, PageNotFoundError, and SearchError to improve clarity and provide meaningful context to users when operations fail. These custom errors ensure consistency with MediaWiki's own API behavior, enabling users to identify and address issues more effectively. For example, when a nonexistent page is queried, the API raises a PageNotFoundError, allowing the user to distinguish this case from other failures.

5. **Revisions Utility Methods**

The revisions property and utility methods represent a significant enhancement to the wrapper's functionality, enabling users to interact with the revisions API in an intuitive and efficient manner. The original wrapper provided no abstraction for revisions, requiring users to construct and parse raw API calls themselves. To address this gap, we designed utility methods that simplify common use cases for fetching, summarizing, and comparing page revisions:

1. **Fetching Recent Revisions**: Retrieves the latest revisions of a page while respecting MediaWiki's rvlimit parameter. Handles batching for requests exceeding the limit, ensuring seamless and compliant data retrieval.
2. **Comparing Revisions**: Abstracts the complexity of constructing revision-specific queries to identify differences between two specific revisions. Useful for tracking content changes in research or journalism.
3. **Summarizing Revisions**: Generates concise summaries of revision activity using metadata such as timestamps, users, and comments. Allows users to quickly assess the significance of recent changes.
4. **Real-Time Monitoring**: Provides live notifications of new revisions by periodically querying the revisions endpoint and analyzing responses, supporting dynamic use cases like tracking high-traffic pages.

These revisions utilities not only streamline access to the MediaWiki revisions API but also demonstrate the flexibility and extensibility of the redesigned wrapper. They highlight our commitment to supporting analytical use cases and empowering users to interact with Wikipedia data in sophisticated ways.

6. **Consistency with the original Wikipedia Restful API**

To align with the original Wikipedia API, we corrected inconsistencies such as the extracts method, which previously returned a full summary instead of the intended sentences. By ensuring that methods faithfully replicate the behavior of their corresponding MediaWiki endpoints, we enhanced the API's reliability and predictability. This consistency is particularly

important for users familiar with the MediaWiki API, as it ensures a seamless transition to the Python wrapper.

7. **Thread Safety**

Concurrent access posed a significant challenge, as the original wrapper lacked thread-safety documentation, with attributes such as WikipediaPage._called being vulnerable to contention during parallel API calls. To address this, we implemented locks to safeguard shared resources, ensuring that the API can handle multiple threads without data corruption or race conditions. Documentation was also updated to outline thread safety considerations, providing users with the confidence to perform concurrent operations.

8. **Pagination**

While pagination of search results was not fully implemented, we made design considerations to address these challenges in the future. Pagination for methods such as section and page retrieval is critical for handling large-scale queries efficiently, and our design lays the foundation for these features, with plans to integrate them seamlessly into the existing framework.

9. **Rate Limiting – Revisions**

While MediaWiki imposes no strict rate limits for its endpoints, it does specify query limits for certain endpoints through the use of special parameters such as rvlimit for revisions endpoint, which caps the number of revisions retrievable in a single request (up to 500 for bot users). For the revisions endpoint and associated utility methods, we implemented mechanisms to respect these limits by batching requests for cases where more than 500 revisions are needed. This ensures compliance with the API's guidelines while avoiding excessive server load.

However, rate limiting was not consistently implemented across all endpoints in the wrapper. For example, methods such as search or content retrieval do not yet include built-in safeguards against excessive request frequencies. Future iterations should incorporate global rate-limiting strategies, potentially using adaptive algorithms to dynamically adjust request intervals based on server responses and API guidelines.

Challenges and Reflections

Challenges Encountered

One of the major challenges we faced during the development process was the evolving nature of the MediaWiki API itself. MediaWiki frequently introduces new parameters or deprecates

existing ones, requiring a flexible yet robust design to accommodate these changes. Balancing the flexibility needed for future-proofing while maintaining a clear and user-friendly interface was a significant design consideration. Additionally, ensuring thread safety during concurrent access posed a technical challenge, as the original wrapper did not address issues like contention for shared resources or race conditions. Implementing locking mechanisms while maintaining high performance required careful engineering and testing.

Another challenge was achieving consistency in error handling. While we aimed for comprehensive error coverage, excessive complexity in error handling could have led to decreased maintainability. Striking a balance between clarity and simplicity was an iterative process that required feedback from testing and user interviews. Furthermore, addressing the lack of documentation in the original wrapper presented difficulties in understanding the edge cases of existing methods and adapting our design to meet user needs effectively.

Achievements

Despite these challenges, we successfully addressed many limitations of the original wrapper. Our enhancements, such as modular class design, custom error handling, and support for advanced use cases, have significantly improved the wrapper's usability and functionality. The implementation of reusable utility functions for common operations, such as fetching revisions or parsing infoboxes, demonstrated our ability to extend the wrapper in a meaningful way. Additionally, our emphasis on user experience—through features like flexible method signatures, intuitive error messages, and consistent naming conventions—ensured that the API is approachable for a wide range of users.

Our integration of features like threading for concurrent access and the revisions property for monitoring page updates highlights the scalability and modernity of our solution. The API is now capable of supporting advanced analytical use cases, making it suitable for both casual users and large-scale applications.

Limitations and Future Work

Scalability

While the wrapper now supports concurrent access, further optimization is needed to handle truly large-scale operations effectively. For instance, fetching massive datasets or processing thousands of pages simultaneously requires additional features such as adaptive rate limiting and more efficient caching strategies. The implementation of pagination for large content retrievals, such as sections and subsections of a page, remains incomplete and would significantly enhance the API's scalability.

Rate Limiting

The current implementation does not consistently account for MediaWiki's rate-limiting guidelines across all endpoints. For high-traffic scenarios, exceeding the rate limits could result

in failed requests or throttled access. Future work should include dynamic rate-limiting mechanisms that adapt to user activity and the API's constraints. Logging and warnings for approaching rate limits could also be implemented to provide real-time feedback to users.

Testing

Although we conducted unit and functional testing for all newly added features, comprehensive testing is an area for further improvement. Stress testing, integration testing, and edge-case analysis need to be expanded to ensure the API remains reliable under all conditions. For example, testing thread safety under heavy loads or handling unusual API responses requires more extensive test coverage.

Future Directions

Considering the size and vastness of scope of the original MediaWiki API, this wrapper could continue to evolve into a more comprehensive tool for interacting with Wikipedia and its associated resources. Features like infobox parsing into structured formats, advanced multilingual analysis, and real-time monitoring of page updates could be further enhanced. Additionally, integrating the wrapper with machine learning models for natural language processing, as demonstrated in our multilingual comparison use case, opens new possibilities for automated content analysis and cross-cultural research.

The wrapper also has potential applications in educational and research settings, where bulk data retrieval and analysis are essential. By addressing the remaining limitations and refining the design, this project could become a widely adopted tool for developers, researchers, and educators.

Work History Summary

The development process for our enhanced Wikipedia API wrapper included several phases, with contributions distributed across team members and tools. While the GitHub repository and project board served as the central repository for version control and code contributions, much of our task assignment, discussions, and iterative refinements were conducted in meetings and documented via Google Docs. As a result, while our GitHub repository reflects the final deliverables, some intermediate design decisions and development history are better represented in supplemental documentation/meeting notes docs attached in the appendix of this report.

Team Contributions

Task	Team Member	API Version
Set up GitHub repo, project and documentation generation	Alanna	v1
Research into existing use cases of mediawiki api	Ellen, Alanna	v1
Sprint 1 User Interviews (wrapper expanded functionality)	Tianyin	v1
Original wrapper's design flaws fixing (class organization, inconsistency, thread-safety, etc.)	Tianyin	v1
Dynamically generated Language enum	Alanna	v1/v2
Search for pages functionality (with kwargs and sort by) and get page functionalities (with fallback search)	Ellen	v1/v2
Sprint 2 User Interviews (client code use cases)	Ellen, Tianyin, Alanna	v2
Concurrent access feature	Tianyin	v2
Revisions functionality & utility methods	Alanna	v2
Client code for new functionalities	Ellen: Flashcard, report, media Tianyin: multilingual comparison LLM Alanna: revisions LLM	v2
Media extraction functionality	Ellen	v2
Custom exceptions and error handling	Ellen	v2
Extract location, infobox, length, redirect, and templates of wikipedia page	Ellen	v2
Unit testing for new features	Ellen, Tianyin, Alanna	v2/final sprint
Final Report	Alanna, tianyin	Final sprint

Design process and timeline

Our project evolved through three major phases, each building on insights from the previous stages to refine and enhance the wrapper's functionality and usability.

Phase 1: Research (v1)

This phase focused on understanding the shortcomings of the original wrapper and conducting preliminary research into MediaWiki API features. Team members familiarized themselves with the existing wrapper's design flaws, identifying pain points such as inconsistent naming, lack of advanced functionality, and poor error handling. Concurrently, we conducted user interviews to gather feedback on desired features and use cases.

Key accomplishments in this phase:

- Set up GitHub repository and documentation tools.
- Conducted interviews and surveys to define user needs.
- Addressed design flaws like misaligned class organization and lack of encapsulation.
- Introduced an initial Language enum for multilingual support.

Phase 2: User Interviews and Scope Refinement (towards v2)

Building on feedback from Phase 1, this phase emphasized refining the scope of our API wrapper to support practical and analytical use cases. Key features like infobox extraction, revision monitoring, and concurrent access were prioritized to address gaps in the original wrapper.

Key accomplishments in this phase:

- Refined use cases based on user feedback (e.g., flashcards, reports, and higher level analytical use cases such as LLM multilingual comparison, concurrent load).
- Added core functionality for concurrent access and advanced search capabilities.
- Enhanced modular class design for scalability and maintainability.
- Developed utility methods for revisions to address dynamic Wikipedia content use cases.

Final Phase: Implementation and Iterative Improvements (v2)

The final phase involved the implementation of advanced features, rigorous testing, and preparation of client code to demonstrate the API's capabilities. Feedback loops from group members, teaching assistants, and interviewees were integral to refining methods and documentation.

Key accomplishments in this phase:

- Implemented and tested features like revisions monitoring and concurrent access.

- Added **kwargs to enable flexibility in API interactions.
- Wrote client code for diverse use cases, showcasing the API's versatility.
- Delivered thorough documentation, including Sphinx autodoc-generated API references.

Conclusion

This project successfully addressed the limitations of the original MediaWiki Python wrapper by enhancing its functionality, usability, and scalability. Through modular design, intuitive error handling, and support for advanced features such as infobox parsing, concurrent access, and multilingual comparisons, our API provides a flexible and user-friendly interface for interacting with Wikipedia. While challenges like thread safety and evolving API parameters required careful design considerations, the final product is robust and extensible, capable of supporting diverse real-world use cases.

Although some areas, such as rate limiting and pagination, remain incomplete, the groundwork has been laid for future development. With further optimization and testing, this API could become a valuable resource for developers and researchers alike. The work done here not only improves upon the original wrapper but also sets the stage for continued innovation in enabling the use of Wikipedia's vast resources for educational, analytical, and cultural applications.

Appendix

Work History

While our GitHub repository accurately represents the final stages of our development process, earlier phases relied more heavily on Google Docs and meetings for task management and scope refinement. This reflects the exploratory nature of our project's beginning, where continuous iterations and feedback loops informed our evolving design. By the later stages, we had transitioned to a more structured workflow on GitHub, allowing for clear milestones and collaborative code reviews.

Relevant Docs:

- V1 Design Doc: API Design Document
- V1 Flaws & Use Cases: Major API flaws + enhanced use cases + client code
- V1 Planning: Planning
- V2 + Final Sprint Planning: Sprint 2 Planning

Artifacts

Original Wikipedia Wrapper API

https://github.com/martin-majlis/Wikipedia-API

Commit History
https://github.com/cmu-api-design/team3-f24/commits/main/

Branches
https://github.com/cmu-api-design/team3-f24/branches

Issue Tracker
https://github.com/cmu-api-design/team3-f24/issues

Pull Request
https://github.com/cmu-api-design/team3-f24/pulls?q=is%3Apr+is%3Aclosed

Documentation
https://cmu-api-design.github.io/team3-f24/wikipediaapi.html